PERFORMANCE ENGINEERING

М. А. Аль-Гунаид

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

по дисциплине
«Тестирование и оценка качества систем»

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ВОЛГОГРАДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

М. А. Аль-Гунаид

ЛАБОРАТОРНЫЙ ПРАКТИКУМ ПО ДИСЦИПЛИНЕ «ТЕСТИРОВАНИЕ И ОЦЕНКА КАЧЕСТВА СИСТЕМ»

Учебно-методическое пособие

Волгоград 2020

УДК 658.012.011.56 (075)

Р е ц е н з е н т ы :
кафедра «Компьютерные технологии и системы»
Брянского государственного технического университета,
зав. кафедрой д-р техн. наук, доцент В. И. Аверченков;

зав. кафедрой «Общая физика»
Астраханского государственного университета
д-р техн. наук, профессор А. М. Лихтер

Печатается по решению редакционно-издательского совета Волгоградского государственного технического университета

М. А. Аль-Гунаид

Даны представления об основных этапах и принципах концептуального проектирования реляционных баз данных. Описаны особенности проектирования баз данных на языке SQL и особенности реализации веб-приложения на языке PHP. Дано представление о шаблоне проектирования MVC (Model – View – Controller), рассмотрена реализация веб-приложения ASP.NET MVC.

Предназначено для студентов вузов, обучающихся по направлению подготовки 09.04.01 «Информатика и вычислительная техника» очной формы обучения.

Ил. 44. Библиогр. : 9 назв.
ISBN 978-5-9948-2442-9

© Волгоградский государственный технический университет, 2020
© М. А. Аль-Гунаид, 2020

Содержание

1 Лабораторная работа №1 **6**
- 1.1 Цели и задачи . . . 6
- 1.2 Критерии приемки работ . . . 6
- 1.3 Выбор предметной области . . . 6
- 1.4 Описание проекта . . . 6
- 1.5 Изучение требований и рекомендаций по оформлению кода . . . 6
- 1.6 Реализация проекта . . . 6
  - 1.6.1 Пример проекта на C# (ASP.Net Core) . . . 6
- 1.7 Пример проекта на Python (Django) . . . 12
- 1.8 Контрольные вопросы . . . 14

2 Лабораторная работа №2 **15**
- 2.1 Цели и задачи . . . 15
- 2.2 Критерии приемки работ . . . 15
- 2.3 Программа и методика испытаний программного обеспечения . . . 15
- 2.4 Реализация Unit-тестов . . . 15
- 2.5 C# (ASP.Net Core) . . . 15
  - 2.5.1 Расположение файлов . . . 15
  - 2.5.2 Unit-тесты . . . 18
  - 2.5.3 Написание Unit-тестов . . . 18
- 2.6 Python (Django) . . . 25
  - 2.6.1 Расположение файлов . . . 25
  - 2.6.2 Unit-тесты . . . 27
  - 2.6.3 Пример написания Unit-тестов . . . 27
- 2.7 Контрольные вопросы . . . 35

3 Лабораторная работа №3 **36**
- 3.1 Цели и задачи . . . 36
- 3.2 Критерии приемки работ . . . 36
- 3.3 Разработка Dockerfile для упаковки разработанной системы в Docker контейне . . . 36
  - 3.3.1 C# (ASP.Net Core) . . . 36
- 3.4 Загрузка собранного контейнера в репозиторий Docker контейнеров . . . 40
- 3.5 Контрольные вопросы . . . 43

4 Лабораторная работа №4 **44**
- 4.1 Цели и задачи . . . 44
- 4.2 Критерии приемки работ . . . 44
- 4.3 Работа с кластером (основные команды) . . . 44
  - 4.3.1 Развертывание системы с помощью Ansible . . . 44
  - 4.3.2 Unit-тесты при развертывании . . . 44
- 4.4 Ручной запуск Unit-тестов в Docker контейнере . . . 45

4.4.1 **Проект на C#** 45
4.4.2 **Проект на Python (Django)** 46
4.5 **Развертывание системы с помощью TFS Piplines** 46
4.6 **Контрольные вопросы** 46
Приложение А **48**
Приложение Б **50**
Приложение В **53**

1 Лабораторная работа №1

Наименование: Выбор предметной области

1.1 Цели и задачи

Целью работы является получение навыков разработки программного кода в соответствии с требованиями, разработанными в приложении Б.
Необходимо выполнить следующие задачи:

1. Выбрать предметную область проекта;
2. Изучить требования и рекомендации по оформлению кода;
3. Описать ход работы;
4. Сформулировать краткий вывод о выполнении работы.

1.2 Критерии приемки работ

1. Реализован проект;
2. Соблюдены требования объектно ориентированного программирования;
3. Соблюдены требования по оформлению кода.

1.3 Выбор предметной области

Предметная область определяется самостоятельно исходя из темы магистерской диссертации.

1.4 Описание проекта

На данном этапе необходимо описать разрабатываемый проект:

- Цели и задачи, которые он решает;
- Сфера применения;
- Язык программирования и технологии, которые будут использоваться при разработке.

1.5 Изучение требований и рекомендаций по оформлению кода

Требования и рекомендации по оформлению кода приведены в приложении Б.

1.6 Реализация проекта

1.6.1 Пример проекта на C# (ASP.Net Core*)*

Исходный код проекта доступен здесь[5].

1. **Создание проекта**
   Создадим новый проект ASP.NET[2] Core по типу Web Application (Model-View-Controller):

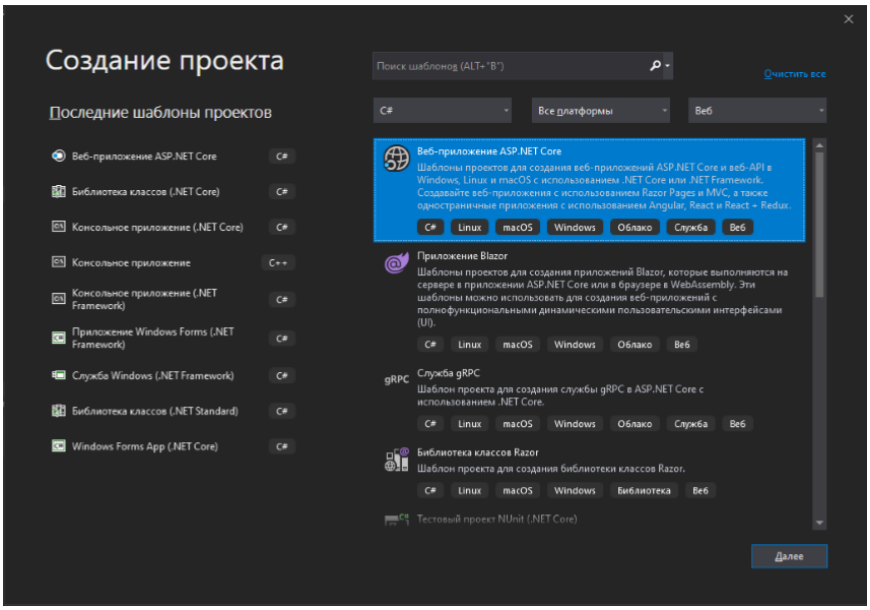

Рис. 1 Создание проекта

Пусть проект будет называться **UnitTestApp**. Проект также понадобится в дальнейшем, при ознакомлении с юнит-тестированием в лабораторной работе №2.

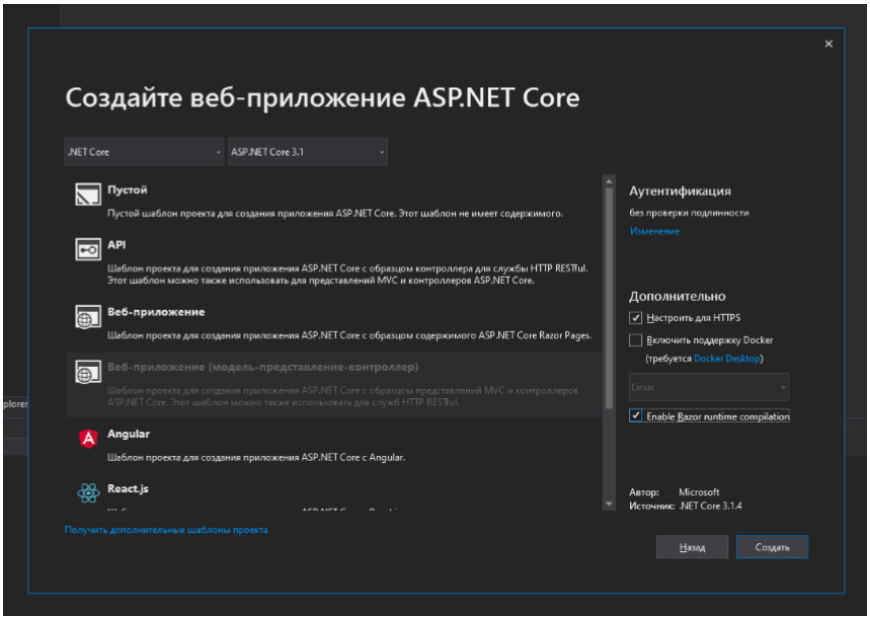

Рис. 2 Создание веб-приложения

2 Реализация функционала проекта

Например, пусть в проекте ASP.NET [1] Core определена модель User:

```
using System.ComponentModel.DataAnnotations;

namespace UnitTestApp.Models
{
    public class User
    {
        public int Id { get; set; }
        [Required]
        [MinLength(1)]
        public string FirstName { get; set; }

        public string LastName { get; set; }

        [Required]
        public int Age { get; set; }

        public string City { get; set; }
    }
}
```

Для работы с данными добавим общий интерфейс репозитория:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace UnitTestApp.Models
{
    public interface IRepository
    {
        IEnumerable<User> GetAll();
        User Get(int id);
        void Create(User user);
        IEnumerable<User> Where(Func<User, bool> func);
    }
}
```

Этот интерфейс может реализоваться разными классами, которые используют разные источники данных - базу данных, файлы xml, список объектов в памяти, внешние сервисы и так далее. Мы реализуем псевдорепозиторий, который работает с "базой данных":

```
using System;
using System.Collections.Generic;
```

```
using System.Linq;

namespace UnitTestApp.Models
{
    public class Repository : IRepository
    {
        // Собственно сама база данных
        // В реальном проекте вместо этого
        будет подключение к базе данных
        private List<User> users = new List<User>()
        {
            new User(){ FirstName="Иван", Age=22, City="Волгоград" },
        };

        // Создание записи User
        public void Create(User user)
        {
            var id = users.Select(p => p.Id).Max() + 1;
            user.Id = id;
            users.Add(user);
        }

        //  Получение пользователя по Id
        public User Get(int id)
        {
            return users.FirstOrDefault(p => p.Id == id);
        }

        // Получения списка всех пользователей
        public IEnumerable<User> GetAll()
        {
            return users;
        }

        // Отбор пользователей из БД с использование пользовательской функции
        public IEnumerable<User> Where(Func<User, bool> func)
        {
            return users.Where(func);
        }
    }
}
```

Добавим наш репозиторий в Dependency Injection в файле *Startup.cs*

```
....
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();
```

```
    // Наш репозиторий
    services.AddSingleton<IRepository, Repository>();

    services.AddMvc().AddRazorRuntimeCompilation();
} ....
```

Теперь, создадим контроллер *Users*:

```
using Microsoft.AspNetCore.Mvc;
using UnitTestApp.Models;

namespace UnitTestApp.Controllers
{
    public class UsersController : Controller
    {
        private readonly IRepository _repository;
        public UsersController(IRepository repository)
        {
            _repository = repository;
        }

        // Страница со списком всех пользователей
        public IActionResult Index() => View(_repository.GetAll());

        // Страница создания пользователя
        public IActionResult Create() => View();

        // Создание пользователя
        [HttpPost]
        public IActionResult Create(User model)
        {
            if (ModelState.IsValid)
            {
                _repository.Create(model);
                return RedirectToAction("Index");
            }
            return View(model);
        }

        // Получение информации о пользователе по Id
        public IActionResult GetUser(string id)
        {
            int intId;
            if (int.TryParse(id, out intId))
            {
                return View(_repository.Get(intId));
```

```
            }
            return View(null);
        }

        //Страница со списком пользователей младше 18 лет
        public IActionResult Less18() => View("Index",
        _repository.Where(p => p.Age < 18));
    }
}
```

Также необходимо создать View файл, для отображения списка пользователей, назовем его *Index*:

```
@model IEnumerable<UnitTestApp.Models.User>

@{
    ViewData["Title"] = "Пользователи";
}

<p>
    <a asp-action="Index">Все пользователи</a>
</p>
<p>
    <a asp-action="Less18">Пользователи младше 18 лет</a>
</p>
<form asp-action="GetUser" method="get">
    <input type="text" name="id" id="id" placeholder="id
    пользователя" />
    <input type="submit" value="Найти пользователя по Id" class="btn
    btn-primary" />
</form>
<a asp-action="Create">Создать пользователя</a>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Id)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.FirstName)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.LastName)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Age)
```

```
            </th>
            <th>
                @Html.DisplayNameFor(model => model.City)
            </th>
        </tr>
    </thead>
     <tbody>
     @foreach (var item in Model)
     {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.Id)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.FirstName)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.LastName)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Age)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.City)
            </td>
        </tr>
    }
    </tbody>
</table>
```

Это лишь часть кода приложения, которая отображает работу с одной моделью и показывает лишь небольшой пример, подробнее можно ознакомиться с исходным кодом в репозитории, найти который можно в списке литературы[6].

1.7 Пример проекта на Python (Django)

* Пример проекта: "Анализ временных рядов полевых культур"можно найти в репозитории из списка литературы[8].

1 **Установка Python и Django**

Для создания Django проекта на Python необходимо установить непосредственно сам Python, для этого на официальном веб-сайте есть инструкция[1].

Для установки Django можно воспользоваться командой для *pip*:

pip install Django

2 Создание проекта

Создание проекта выполняется при помощи консольной команды:
django-admin startproject UPE_unit_tests_app .
где *UPE_unit_tests_app* - название создаваемого проекта,*django-admin* - скрипт, который создаст директорий и файлы для проекта.
Запуск программы выполняется из корневой папки проекта при помощи консольной команды:
python manage.py runserver
По-умолчанию, для просмотра сайта в любом браузере нужно перейти по следующему адресу: *http://127.0.0.1:8000/*.

3 Реализация функционала проекта

Фреймворк Django также использует шаблон проектирования MVC.
В файле *models.py* опишем модели:

```
from django.db import models

class Request(models.Model):
    datetime = models.DateTimeField()
    rmse = models.FloatField()
    mae = models.FloatField()
    mape = models.FloatField()

class CSVFile(models.Model):
    namefile = models.TextField()
    filecontent = models.TextField()
    countrows = models.IntegerField()
    request = models.OneToOneField(Request,
    on_delete=models.CASCADE, primary_key=True)

class Monitoring(models.Model):
    memory = models.FloatField()
    cpu = models.FloatField()
    time = models.FloatField()
    request = models.OneToOneField(Request,
    on_delete=models.CASCADE, primary_key=True)
```

Файл *views.py* представляет собой контроллер, пример кода:

```
from django.http import *
from django.views.decorators.csrf import csrf_exempt
from BI.models import *

@csrf_exempt
def Hello(request: HttpRequest):
    return HttpResponse("General Kenobi!")
```

Также необходимо настроить маршрутизацию адресов, для этого существует файл *urls.py*, в который внесем следующий код:

```
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static
from BI import views

urlpatterns = [
     #прогнозирование на данных из файла
     path('predict/', views.Predict),
     #запрос для проверки работоспособности
     path('helloThere/', views.Hello),

     path('request/', views.AutoRequest),

    ] + static(settings.STATIC_URL,  document_root
    =settings.STATIC_ROOT) + static("/log/", document_root="]
```

Теперь при переходе по адресу *http://127.0.0.1:8000/helloThere* будет выполняется функция Hello из файла *views.py*.

Это лишь часть кода приложения, которая кратко показывает работу с Django и показывает лишь небольшой пример, подробнее можно ознакомиться с исходным кодом в репозитории, из списка литератры[9].
Подробнее с особенностями Django можно ознакомиться в официальной документации[3].

1.8 Контрольные вопросы

Список вопросов для отчета:

1. Что такое ООП? Дайте расшифровку и краткое описание.
2. Что такое MVC? Дайте расшифровку и краткое описание.
3. Какие источники данных используют различные классы, реализованные приведенным интерфейсом?
4. Как реализовать псевдо-репозиторий, работающий с "базой данных"?
5. Какой шаблон использует фреймворк Django?

2 Лабораторная работа №2

Наименование: Программа и методика испытаний

2.1 Цели и задачи

Целью работы является получение навыков разработки программы и методики испытаний программного обеспечения, а также реализации Unit-тестов.
Необходимо выполнить следующие задачи:

1. Разработать программу и методику испытаний программного обеспечения;
2. Оформить документ программа и методика испытаний программного обеспечения в соответствии с требованиями ГОСТ 34.603-92 РД 50-34.698-90;
3. Верифицировать разработанные сценарии в программе и методике испытаний программного обеспечения с использованием Unit-тестов.

С примером документа **Программа и методика испытаний программного обеспечения** можно ознакомиться в приложении А .

2.2 Критерии приемки работ

1. Разработана программа и методика испытаний программного обеспечения;
2. Разработаны и описаны Unit-тесты;
3. Unit-тесты соответствуют требованиям разработанным приложении Б.

2.3 Программа и методика испытаний программного обеспечения

Существует множество сценариев тестирования[3], вот основные из них:

- TDD — Test Driven Development
- BDD — Behaviour Driven Development
- TDD — Type Driven Development
- DDD — Domain Driven Design
- FDD — Features Driven Developmen
- MDD — Model Driven Developmen
- PDD — Panic Driven Development

2.4 Реализация Unit-тестов

Разрабатываемые системы будут упаковываться в Docker контейнера и разворачиваться на удаленных машинах, следовательно, необходимо придерживаться стандарта расположения файлов Dockerfile и docker-compose в проекте.

2.5 C# (ASP.Net Core)

2.5.1 Расположение файлов

Файлы для конфигурации Docker образов должны храниться в корне проекта. Для не большого проекта достаточно 4х файлов:

1 .dockerignore
2 .env
3 NAME_SERVICE.Dockerfile
4 docker-compose.yml

Следовательно, файлы конфигурации Docker должны располагаться в корне проекта:

1 PROJECT_NAME/.dockerignore
2 PROJECT_NAME/.env
3 PROJECT_NAME/NAME_SERVICE.Dockerfile
4 PROJECT_NAME/docker-compose.yml

Описание файлов:

1 *.dockerignore* - файл содержащий шаблоны файлов и каталогов для их исключения во время сборки Docker контейнера. Используется для повышения производительности и понижения размера собранного образа.
Пример:
.dockerignore
.env
.git
.gitignore
.vs
.vscode
**/bin*
**/obj*
***/.toolstarget*

2 *.env* - содержит список переменных, используется только во время стадии предварительной обработки docker-compose
Пример:
$ cat .env
TAG=v1.5
$ cat docker-compose.yml

```
version: '3'
services:
  web:
    image: "webapp:\${TAG}"
```

3 NAME_SERVICE.Dockerfile - представляет собой текстовый документ, который содержит все команды, которые пользователь может вызвать в командной строке для сборки образа. **На примере "проект-пример с Unit-тестами":**

```
FROM mcr.microsoft.com/dotnet/core/sdk:3.1 AS base
WORKDIR /app
EXPOSE 80

FROM mcr.microsoft.com/dotnet/core/sdk:3.1 AS build

WORKDIR /src
COPY ["UnitTestApp/UnitTestApp.csproj", "UnitTestApp/"]
COPY ["UnitTestApp.Tests/UnitTestApp.Tests.csproj",
"UnitTestApp.Tests/"]
RUN dotnet restore "UnitTestApp/UnitTestApp.csproj"

COPY . .
WORKDIR "/src"
RUN dotnet build "UnitTestApp/UnitTestApp.csproj" -c Release -o /app
RUN dotnet build
"UnitTestApp.Tests/UnitTestApp.Tests.csproj" -c Release -o /app

FROM build AS publish
RUN dotnet publish "UnitTestApp/UnitTestApp.csproj" -c  Release -o /app
RUN dotnet publish
"UnitTestApp.Tests/UnitTestApp.Tests.csproj" -c Release -o /app

# Запуск тестов
RUN dotnet test
"UnitTestApp.Tests/UnitTestApp.Tests.csproj"

FROM base AS final
WORKDIR /app
COPY --from=publish /app .
ENTRYPOINT ["dotnet", "UnitTestApp.dll"]
```

4 *docker-compose.yml* - это инструмент для определения и запуска многоконтейнерных приложений Docker. С Compose вы используете файл YAML для настройки служб вашего приложения. Затем с помощью одной команды вы создаете и запускаете все службы из вашей конфигурации. **На примере "проект-пример с Unit-тестами":**

```
version: '3.4'

services:
  unittestapp:
    image: "unittestapp:\${TAG}"
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    build:
       context: ./
       dockerfile: UnitTestApp.Dockerfile
```

```
        ports:
        - 80:80
```

2.5.2 Unit-тесты

В проекте должны быть реализовано тестирование, оно происходит при каждой сборке Docker контейнера.
Пример проекта с Unit-тестами доступен в репозитории из списка литературы[7].
ссылке
Dockerfile должен содержать инструкцию по запуску тестов при сборке:

```
... RUN dotnet publish
"UnitTestApp.Tests/UnitTestApp.Tests.csproj" -c Release -o /ap

# Запуск тестов RUN dotnet test
"UnitTestApp.Tests/UnitTestApp.Tests.csproj"

FROM base AS final
...
```

2.5.3 Написание Unit-тестов

Исходный код проекта доступен в репозитории из списка литературы[7]

1 **Создание проекта**
   В лабораторной работе №1 мы уже создавали проект C# (ASP.Net Core[1]).
   Теперь для него добавим проект юнит-тестов.
   Название проекта юнит-тестов оформляется как [название_приложения].Tests.
   Итак, добавим в решение новый проект по типу Class Library (.NET Core):

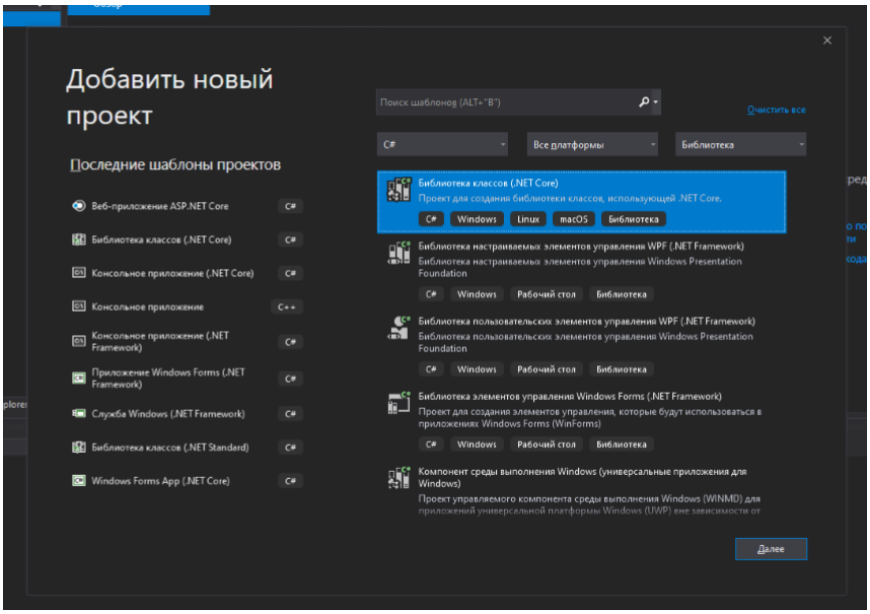

Рис. 3 Создание приложения

Назовем его **UnitTestApp.Tests**

Начиная с Visual Studio 2019, были добавлены следующие типы проектов для тестов: **xUnit Test Project(.NET Core), MSTest Test Project(.NET Core) и NUnit Test Project(.NET Core).** Но мы добавим все зависимости вручную.

После создания проектов, мы имеем следующую структуру решения:

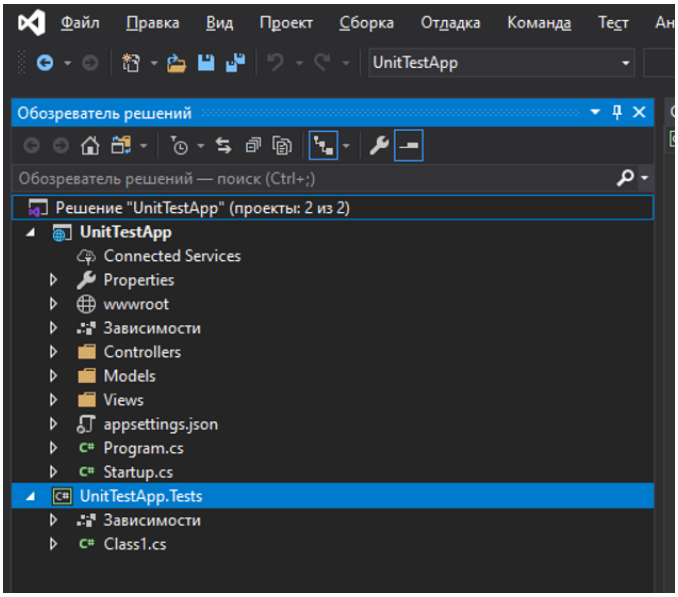

Рис. 4 Структура решения

Удалим из проект тестов созданный по умолчанию файл **Class1.cs**, так как он нам не нужен.

2 **Добавление фреймворка для Unit-тестов**

Теперь добавим фреймворк Unit-тестов. Существует множество фреймворков. В данном случае мы будем использовать xUnit как наиболее распространенный фреймворк в связке с ASP.NET Core [1] (можно использовать и другие фреймворки).

Чтобы использовать xUnit, в проект тестов (то есть UnitTestApp.Tests) необходимо добавить через NuGet четыре пакета:

- **Microsoft.NET.Test.Sdk**
- **xunit:** функционал фреймворка тестирования
- **xunit.runner.visualstudio:** обеспечивает интеграцию с Visual Studio
- **Moq**

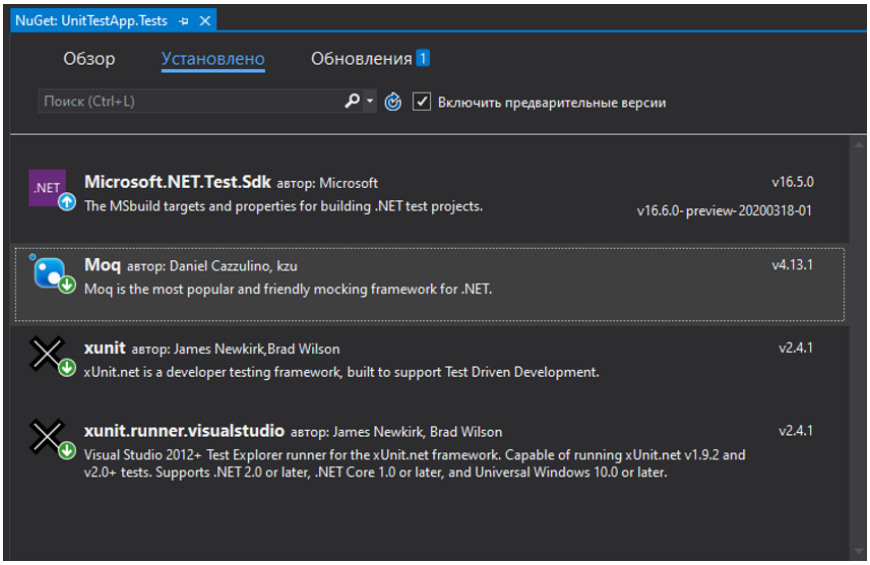

Рис. 5 Добавление фрейморка

После добавления этих пакетов также в проекте тестов необходимо добавить ссылку на главный проект:

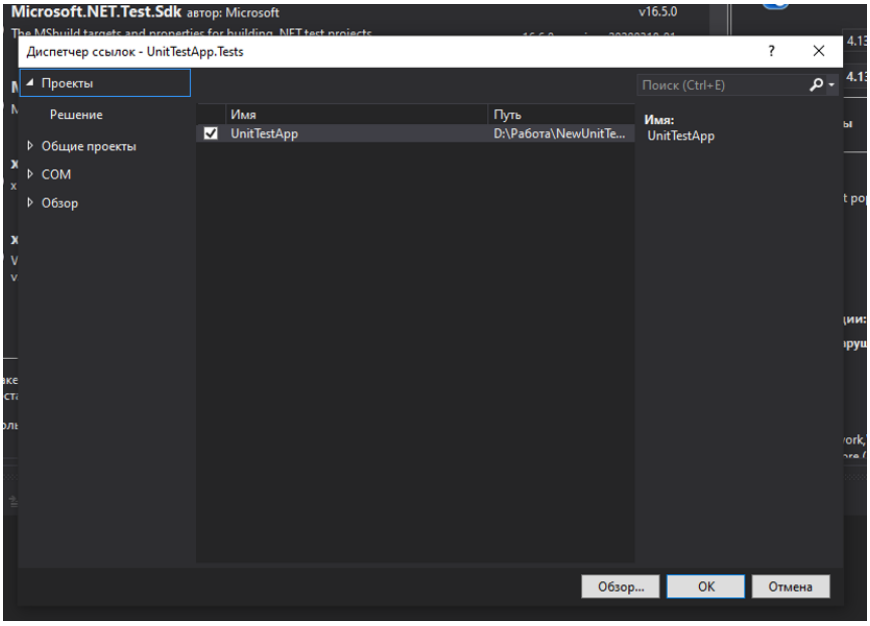

Рис. 6 Диспетчер ссылок UnitTestApp.Tests

Таким образом, проект тестов настроен, и мы можем приступать к тестированию.

3 **Реализация функционала проекта**
Для тестирования будем использовать функционал на примере модели *User*, создание которой было описано в лабораторной работе №1.

4 **Unit-тесты** В проекте *UnitTestApp.Tests* создадим файл *UsersControllerTest.cs* и напишем несколько тестов к Users контроллеру:

```
using Microsoft.AspNetCore.Mvc;
using Moq;
using System.Collections.Generic;
using System.Linq;
using UnitTestApp.Controllers;
using UnitTestApp.Models;
using Xunit;

namespace UnitTestApp.Tests.Controllers
{
    public class UsersControllerTest
    {
        private List<User> GetAllFakeUsers()
        {
```

```
        //Имена пользователей вымышлены, любые совпадения
        чистая случайность.
        return new List<User>()
        {
            new User(){ Id=1, FirstName="Алеша", LastName="Попович",
            Age=20, City="Русь" },
            new User(){ Id=2, FirstName="Илья", LastName="Муромец",
            Age=33, City="Русь" },
            new User(){ Id=3, FirstName="Добрыня", LastName="Никитич",
            Age=35, City="Русь" },
        };
    }
/// <summary>
/// Тестируем Action - [Index]
/// Сценарий - получение результата выполнения контроллера (ViewResult)
/// Результат - View с данными типа "IEnumerable<User>"
/// </summary>
[Fact]
public void Index_ViewResult_ViewResultAndListEntities()
{
    // Arrange
    // Заменяем метод "реального" репозитория который работает с БД
    // это делается для того что бы ничего не сломать в рабочих данных
    var mock = new Mock<IRepository>();
    mock.Setup(repo => repo.GetAll()).Returns(GetAllFakeUsers());
    var controller = new UsersController(mock.Object);

    // Act
    var result = controller.Index(); // Результат контроллера

    // Assert
    // Действительно ли контроллер возвращ
    var viewResult = Assert.IsType<ViewResult>(result);
    // Проверяет, чт
    var model = Assert.IsAssignableFrom<IEnumerable<User>>(viewResult.Model);
    // Кол-во данных совпадает и контрол
    Assert.Equal(GetAllFakeUsers().Count, model.Count());
}

 /// <summary>
 /// Тестируем Action - [Create]
 /// Сценарий - создание пользователя с пустым именем
 /// Результат - View с заполнеными полями и вывод
 ///сообщение об ошибках при заполнении формы
 /// </summary>
 [Fact]
```

```
 public void Create_CreateUserWithEmptyFirstName_ViewResult()
 {
    // Arrange
    var mock = new Mock<IRepository>();
    var controller = new UsersController(mock.Object);
    controller.ModelState.AddModelError("FirstName", "Required");
    User newUser = new User();

    // Act
    var result = controller.Create(newUser);

    // Assert
    var viewResult = Assert.IsType<ViewResult>(result);
    Assert.Equal(newUser, viewResult?.Model);
}

/// <summary>
/// Тестируем Action - [Create]
/// Сценарий - создание пользователя
/// Результат - RedirectToActionResult
/// </summary>

[Fact]
public void Create_CreateUser_RedirectToActionResult()
{
    // Arrange
    var mock = new Mock<IRepository>();
    var controller = new UsersController(mock.Object);
    var newUser = new User()
        {
            FirstName = "Ben"
        };

    // Act
    var result = controller.Create(newUser);

    // Assert
    var redirectToActionResult = Assert.IsType<RedirectToActionResult>(result);
    Assert.Null(redirectToActionResult.ControllerName);
    Assert.Equal("Index",redirectToActionResult.ActionName);
    // Проверяем, был вызван ли метод Create у репозитори
    mock.Verify(r => r.Create(newUser));
}

/// <summary>
/// Тестируем Action - [GetUser]
```

```
    /// Сценарий - Плохой запрос, с неверными данными
    /// Результат - ViewResult с пустой Model
    /// </summary>
    [Fact]
    public void GetUser_BadRequest_ViewResultWithEmptyModel()
    {
        // Arrange
        var mock = new Mock<IRepository>();
        var controller = new UsersController(mock.Object);

        // Act
        var result = controller.GetUser(null);

        // Arrange
        var viewResult = Assert.IsType<ViewResult>(result);
        Assert.Null(viewResult?.Model);
    }

    /// <summary>
    /// Тестируем Action - [GetUser]
    /// Сценарий - Получение пользователя по Id
    /// Результат - ViewResult с данными пользователя
    /// </summary>
    [Fact]
    public void GetUser_GetUserById_ViewResultWithUser()
    {
        // Arrange
        int testUserId = 1;
        var mock = new Mock<IRepository>();
        mock.Setup(repo => repo.Get(testUserId)) .Returns(GetAllFakeUsers()
        .FirstOrDefault(
        p => p.Id == testUserId));
        var controller = new UsersController(mock.Object);

        // Act
        var result =controller.GetUser(testUserId.ToString());
        // Assert
        var viewResult = Assert.IsType<ViewResult>(result);
        var model = Assert.IsType<User>(viewResult.ViewData.Model);
        Assert.Equal("Алеша", model.FirstName);
        Assert.Equal("Попович", model.LastName);
        Assert.Equal("Русь", model.City);
        Assert.Equal(20, model.Age);
        Assert.Equal(testUserId, model.Id);
    }
  }
}
```

После запуска, мы видим, что все тесты пройдены успешно:

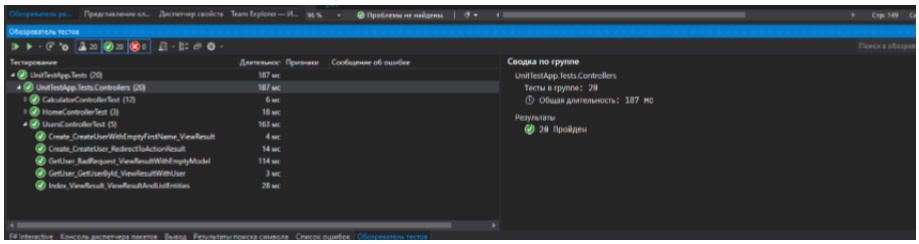

Рис. 7 Просмотр результата

2.6 Python (Django)

Пример проекта: "Анализ временных рядов полевых культур"содержащий Unit-тесты доступен в репозитории из списка литературы[8].

2.6.1 Расположение файлов

Файлы для конфигурации Docker образов должны храниться в корне проекта. Для не большого проекта достаточно 4х файлов:

1 .dockerignore
2 .env
3 NAME_SERVICE.Dockerfile
4 docker-compose.yml

Следовательно, файлы конфигурации Docker должны располагаться в корне проекта:

1 PROJECT_NAME/.dockerignore
2 PROJECT_NAME/.env
3 PROJECT_NAME/NAME_SERVICE.Dockerfile
4 PROJECT_NAME/docker-compose.yml

Описание файлов:

1 *.dockerignore* - файл содержащий шаблоны файлов и каталогов для их исключения во время сборки Docker контейнера. Используется для повышения производительности и понижения размера собранного образа.
Пример:
.dockerignore
.env
.git
.gitignore
.vs

.vscode
**/bin*
**/obj*
***/.toolstarget*

2 *.env* - содержит список переменных, используется только во время стадии предварительной обработки docker-compose **Пример**:
$ cat .env
TAG=latest
$ cat docker-compose.yml

```
version: '3'
services:
    web:
      image: "webapp:\${TAG}"
```

3 *NAME_SERVICE.Dockerfile* - представляет собой текстовый документ, который содержит все команды, которые пользователь может вызвать в командной строке для сборки образа. **На примере "Анализ временных рядов полевых культур содержащий Unit-тесты":**

```
FROM python:3.6
ENV PYTHONUNBUFFERED 1
COPY . .

RUN pip install -r requirements.txt
EXPOSE 8000

# Запуск тестов
RUN python manage.py test

CMD [ "python", "manage.py", "runserver", "0.0.0.0:8000" ]
```

4 *docker-compose.yml* - это инструмент для определения и запуска многоконтейнерных приложений Docker. С Compose вы используете файл YAML для настройки служб вашего приложения. Затем с помощью одной команды вы создаете и запускаете все службы из вашей конфигурации. **На примере "Анализ временных рядов полевых культур содержащий Unit-тесты":**

```
version: '3'

services:
    web:
        build: .
        command: python3 manage.py runserver 0.0.0.0:8000
        volumes:
           - .:/code
```

```
        ports:
          - "8000:8000"
```

2.6.2 Unit-тесты

В проекте должны быть реализовано тестирование, оно происходит при каждой сборке Docker контейнера.
Пример проекта: "Анализ временных рядов полевых культур"содержащий Unit-тесты доступен в репозитории из списка литературы[8]
Dockerfile должен содержать инструкцию по запуску тестов при сборке:

```
...
EXPOSE 8000

# Запуск тестов
RUN python manage.py test

CMD [ "python", "manage.py", "runserver", "0.0.0.0:8000" ]
...
```

2.6.3 Пример написания Unit-тестов

Весь код проекта доступен репозитории из списка литературы[7].

1 **Возьмем готовый проект анализ временных рядов посевных культур**
В файле */BI/views.py* есть метод Predict который доступен после запуска приложения по адресу: *http://IP_ADDRESS/predict/*

```
...
def Predict(request: HttpRequest):
dateTime = datetime.utcnow()
# Засекаем время начала обработки запроса
start_time = time.clock()
# Засекаем как сильно загрузился процессор во время обработки заброса
psutil.cpu_percent(interval=None, percpu=True)
try:
    data = []
    if len(request.FILES) == 1:
        # Имя обрабатываемого файла можно найти как request.FILES['file']
        for key in request.FILES:
            csvfile = GetFile(key, request.FILES[key].file, request.encoding)
            data = handle_files(csvfile.filecontent)
    else:
        return HttpResponseServerError("<h2>No file</h2>")
    try:
        date = [y[0] for y in data]

        revenue = [float(y[1]) for y in data if (y[1] is not "")]
        crop = [float(y[2]) for y in data if (y[2] is not "")]
        temp = [float(y[3]) for y in data if (y[3] is not "")]
```

```
rain = [float(y[4]) for y in data if (y[4] is not "")]
cost = [float(y[5]) for y in data if (y[5] is not "")]

except Exception as e:
    return HttpResponseServerError("<h2>Wrong file: "
    + str(e) + "</h2>")

crop_forecast_rf = CropForecast(crop, temp, rain, "RF")
cost_forecast_rf = CostForecast(cost, crop +
crop_forecast_rf[len(crop):], temp, rain, "RF")
revenue_forecast_train_rf, revenue_forecast_test_rf =
RevenueForecast(revenue,cost + cost_forecast_rf[
        crop + crop_forecast_rf[
        rain,
        "RF")

rf_rmse = Forecast.RMSE(revenue, (revenue_forecast_train_rf +
revenue_forecast_test_rf)[0:len(
rf_mae = Forecast.MAE(revenue, (revenue_forecast_train_rf +
revenue_forecast_test_rf)[0:len(re
rf_mape = Forecast.MAPE(revenue, (revenue_forecast_train_rf +
revenue_forecast_test_rf)[0:len(

crop_forecast_lr = CropForecast(crop, temp, rain, "LR")
cost_forecast_lr = CostForecast(cost, crop
+ crop_forecast_lr[len(crop):],
temp, rain, "LR")
revenue_forecast_train_lr, revenue_forecast_test_lr =
RevenueForecast(revenue,
    cost + cost_forecast_lr[
    crop + crop_forecast_lr[
    rain,
    "LR")
lr_rmse = Forecast.RMSE(revenue, (revenue_forecast_train_lr +
revenue_forecast_test_lr)[0:len(
lr_mae = Forecast.MAE(revenue, (revenue_forecast_train_lr +
revenue_forecast_test_lr)[0:len(re
lr_mape = Forecast.MAPE(revenue, (revenue_forecast_train_lr +
revenue_forecast_test_lr)[0:len

response = {
    'date': [y for y in date],
    'revenue': {
        'fact': [y for y in revenue],
        'forecast': [y for y in
        revenue_forecast_train_rf +
```

```
            revenue_forecast_test_rf],
            'RMSE': [rf_rmse],
            'MAE': [rf_mae],
            'MAPE': [rf_mape]
        },
        'crop': {
            'fact': [y for y in crop],
            'forecast': [y for y in crop_forecast_rf]
        },
        'cost': {
            'fact': [y for y in cost],
            'forecast': [y for y in cost_forecast_rf]
        },
        'temp': [y for y in temp],
        'rain': [y for y in rain]
    }
    # Засекаем время конца обработки запроса

    end_time = time.clock()
    memory_usage = 123  # res.group(1)
    # Нахожу сколько весит массив с данными (по идее
    #столько же сколько файл)
    data_size = sys.getsizeof(data)
    # Как сильно был загружен процессор с момента
    #последнего запроса команды
    cpu_percent = psutil.cpu_percent(interval=None, percpu=False)

    # Логирую в файл данные для дальнейшей обработки
    file = open("log.txt", "a")
    file.write(make_scv_str(data, memory_usage,
    end_time - start_time, cpu_percent))
    monitoring = {
        'cpu': cpu_percent,
        'memory': memory_usage,
        'time': end_time - start_time
    }

    file.close()

    empty_response = {
        'metrics': {
            'RMSE': [rf_rmse],
            'MAE': [rf_mae],
            'MAPE': [rf_mape]
        }
    }
```

```
            SaveToDB(empty_response['metrics'], dateTime,
            csvfile, monitoring)
            return HttpResponse(json.dumps(empty_response))
        except Exception as e:
            return HttpResponseServerError("<h2>An error
            occured " + str(e) + "</h2>")
```

Он является основным методом приложения. В него передается файл с обучающими данными (за прошедший период), на основе которого строится прогноз на будущие продажи.

Пример файла с данными (CSV формат):

```
...
2162-02-01,0,17.6,5.4,25,0
2162-03-01,7672040,17.6,-2.5,72,0
2162-04-01,3680358,17.6,-3.7,28,15
2162-05-01,0,17.6,-7.3,2,0
2162-06-01,0,17.6,-4,39,9.7
2162-07-01,0,18,1.3,56,0
2162-08-01,0,18,9.3,13,14.96
2162-09-01,668766,18,16.7,35,15.6
2162-10-01,7116103,18,24,41,15
2162-11-01,9793751,18,24.1,25,0
2162-12-01,2875759,18,23,72,0
2163-01-01,0,18,20.1,28,15
2163-02-01,10035626,18,5.8,2,0
2163-03-01,6051600,18,3.3,39,9.7
2163-04-01,900000,18,-0.2,44,9.5
2163-05-01,0,18,-7.4,16,9
2163-06-01,0,18,0.4,45,9.86
2163-07-01,3180600,20.2,3.4,32,9.26
2163-08-01,0,20.2,11.3,23,9.8
2163-09-01,168198,20.2,16.3,40,0
2163-10-01,2173372,20.2,21.6,15,0
2163-11-01,5562870,20.2,24.5,25,12.66
...
```

2 Написание Unit-тестов

Создадим папку *tests* и поместим её в директорию */BI/* Файлы содержащие Unit-тесты должны начинаться с префикса *test_*, создадим 2 файла:

- *test_views.py* - содержит тесты для проверки ответов сервера на запросы
- *test_DB.py* - содержит тесты для тестирования работы с базой данных

Напишем следующие тесты, для проверки ответов сервера на запросы, в файле test_*views.py*:

```
from django.test import TestCase
from django.core.files.uploadedfile import SimpleUploadedFile
from BI.models import *
from datetime import datetime

class ViewsTest(TestCase):

    @classmethod
    def setUpTestData(cls):
        pass

    def setUp(self):
        pass

    '''
    - Тестируется Get запрос по адресу: [/helloThere/]
    - Проверяется код ответа и содержимое
    - Код результата должен быть [200] и содержать строку ["General Kenobi!"]
    '''

    def test_Hello_MethodGet_SuccessCode(self):
    # Arrange

    # Act
    response = self.client.get("/helloThere/")

    # Assert
    self.assertEqual(200, response.status_code)
    self.assertEqual(b"General Kenobi!", response.content)

    '''
    - Тестируется Get запрос по адресу: [/predict/]
    - Проверяется код ответа без передачи параметров
    - Код результата должен быть [500] и содержать информацию об ошибке
    '''

    def test_Predict_MethodGetWithEmptyRequestHeaders_InternalServerErrorCode(self):
    # Arrange

    # Act
    response = self.client.get("/predict/")

    # Assert
    self.assertEqual(500, response.status_code)
```

```
self.assertNotEqual(b"", response.content)

'''
- Тестируется Post запрос по адресу: [/predict/]
- Проверяется код ответа     - Код результата должен быть
[200] и содержать информацию и выполнении метода
'''

def test_Predict_MethodGetWithFile_SuccessCode(self):
# Arrange

# Act
with open('Data.csv', 'rb') as f:
response = self.client.post("/predict/", {'test.csv': f})

# Assert
self.assertEqual(200, response.status_code)
self.assertNotEqual(b"", response.content)

'''
- Тестируется Post запрос по адресу: [/predict/]
- Проверяется запись в базу данных
- Код результата должен быть [200] и количество записей
увеличиться на 1 единицу
'''

 def test_Predict_SaveRequestToDB_NumberEntities(self):
       # Arrange
       # Количество записей в базе данных до отправки запроса

     beforeNumberRequests = len(Request.objects.all())
     beforeNumberCSVFiles = len(CSVFile.objects.all())
     beforeNumberMonitorings =  len(Monitoring.objects.all())

      # Act
      with open('Data.csv', 'rb') as f:
          response = self.client.post("/predict/", {'test.csv': f})

     # Количество записей в базе данных после исполнения запроса
     afterNumberRequests = len(Request.objects.all())
     afterNumberCSVFiles = len(CSVFile.objects.all())
     afterNumberMonitorings =len(Monitoring.objects.all())

     # Assert
     self.assertEqual(200, response.status_code)
     self.assertEqual(beforeNumberRequests + 1,
```

```
        afterNumberRequests)
        self.assertEqual(beforeNumberCSVFiles + 1,
        afterNumberCSVFiles)
        self.assertEqual(beforeNumberMonitorings + 1,
        afterNumberMonitorings)
    ...
```

Напишем следующие тесты, для проверки работы с базой данных, в файле *test_DB.py:*

```
from django.test import TestCase
from BI.models import *
from datetime import datetime

class DBTest(TestCase):

    @classmethod
    def setUpTestData(cls):
        pass
    def setUp(self):
        pass
    '''
    - Тестируется база данных
    - Проверяется запись в базу данных сущность типа [Request]
    - Успешная запись в базу данных
    '''

    def test_DB_SaveEntityRequestToDB_SuccessSaveToDB(self):
        # Arrange
        request = Request()
        request.rmse = 1
        request.mape = 2
        request.mae = 3
        request.datetime = datetime.utcnow()

        # Act
        request.save()
        entityFromDb = Request.objects.last()

        # Assert
        self.assertEqual(request.mae, entityFromDb.mae)
        self.assertEqual(request.mape, entityFromDb.mape)
        self.assertEqual(request.rmse, entityFromDb.rmse)

    '''
    - Тестируется база данных
    - Проверяется запись в базу данных сущность типа [CSVFile]
```

```
- Успешная запись в базу данных
'''

def test_DB_SaveEntityCSVFileToDB_SuccessSaveToDB(self):
    # Arrange
    csvfile = CSVFile()
    csvfile.filecontent = "1,2,3,4"
    csvfile.countrows = 1
    csvfile.namefile = "namefile.csv"

    # Act
    csvfile.save()
    entityFromDb = CSVFile.objects.last()

    # Assert
    self.assertEqual(csvfile.filecontent,
    entityFromDb.filecontent)
    self.assertEqual(csvfile.countrows,
    entityFromDb.countrows)
    self.assertEqual(csvfile.namefile,
    entityFromDb.namefile)

 '''
 - Тестируется база данных
 - Проверяется запись в базу данных сущность типа [Monitoring]
 - Успешная запись в базу данных
 '''
def test_DB_SaveEntityMonitoringToDB_SuccessSaveToDB(self):
# Arrang

monitoring = Monitoring()
monitoring.memory = 12512.124
monitoring.cpu = 0.5
monitoring.time = 100.531

# Act
monitoring.save()
entityFromDb = Monitoring.objects.last()

# Assert
self.assertEqual(monitoring.memory, entityFromDb.memory)
self.assertEqual(monitoring.cpu, entityFromDb.cpu)
self.assertEqual(monitoring.time, entityFromDb.time)
```

После запуска, мы видим, что все тесты пройдены успешно и процесс завершился с кодом 0:

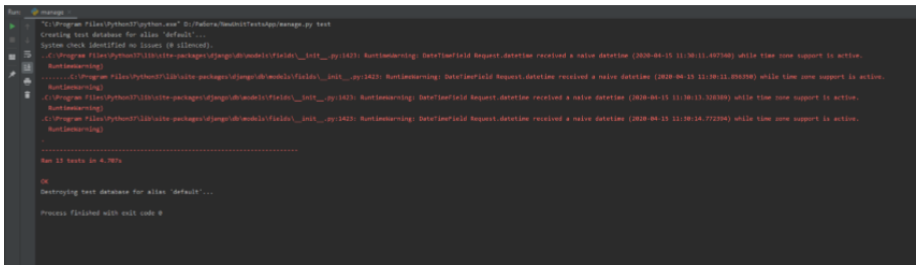

Рис. 8 Успешное завершение тестов

2.7 Контрольные вопросы

Список вопросов для отчета:

1. Зачем нужно тестирование ПО?
2. Что такое Unit-тесты?
3. Какие существуют виды сценариев тестирования?
4. Как правильно выполнять тестирование функций для работы с БД?
5. Что такое тестовый сценарий? Приведите пример.

3 Лабораторная работа №3

Наименование: Docker

3.1 Цели и задачи

Целью работы является ознакомление с технологией Docker и практическое применение навыков на примере разрабатываемого приложения.
Необходимо выполнить следующие задачи:
1. Изучить технологию Docker;
2. Разработка Dockerfile для упаковки разработанной системы в Docker контейнер;
3. Загрузка собранного контейнера в репозиторий Docker контейнеров;

3.2 Критерии приемки работ

1. Собран контейнер;
2. Собранный контейнер залит в репозиторий Docker;

3.3 Разработка Dockerfile для упаковки разработанной системы в Docker контейне

3.3.1 C# (ASP.Net Core)

1 Сначала необходимо создать ASP.Net Core приложение, но мы возьмем приложение, которое создали в лабораторной работе №1 в разделе Пример проекта на C# (ASP.Net Core).После создания зайдем в файл Properties/launchSettings.json, он должен содержать следующие поля:

```
1  {
2    "iisSettings": {
3      "windowsAuthentication": false,
4      "anonymousAuthentication": true,
5      "iisExpress": {
6        "applicationUrl": "http://localhost:59851",
7        "sslPort": 44338
8      }
9    },
10   "profiles": {
11     "IIS Express": {
12       "commandName": "IISExpress",
13       "launchBrowser": true,
14       "environmentVariables": {
15         "ASPNETCORE_ENVIRONMENT": "Development",
16         "ASPNETCORE_HOSTINGSTARTUPASSEMBLIES": "Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation"
17       }
18     },
19     "UnitTestApp": {
20       "commandName": "Project",
21       "launchBrowser": true,
22       "applicationUrl": "http://localhost:5000",
23       "environmentVariables": {
24         "ASPNETCORE_ENVIRONMENT": "Development",
25         "ASPNETCORE_HOSTINGSTARTUPASSEMBLIES": "Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation"
26       }
27     }
28   }
29 }
30
```

Рис. 9 Содержимое файла launchSettings.json

Нас интересует секция UnitTestApp (название секции совпадающее с названием приложения), а именно поле applicationUrl. Данное поле указыва-

ет на каком адресе и порту будет доступно веб приложение после запуска. В дальнейшем, нам пригодится только значение порта.

2 **Создание образа Docker**

Для того чтобы докер собрал нам образ(image), нам надо рассказать ему[докеру] как это сделать по средством комманд записанных в докер файле.

Докер создает обзар(image) читая команды из Dockerfile.Создами Dockerfile с и назовем его *UnitTestApp.Dockerfile* в корне проекта и наполним его следующими командами:

```
FROM mcr.microsoft.com/dotnet/core/sdk:3.1 AS sdk
WORKDIR /app
EXPOSE 80

FROM sdk AS build
WORKDIR /src
COPY ["UnitTestApp/UnitTestApp.csproj", "UnitTestApp/"]
COPY ["UnitTestApp.Tests/UnitTestApp.Tests.csproj", "UnitTestApp.Tests/"]
RUN dotnet restore "UnitTestApp/UnitTestApp.csproj"

COPY . .
FROM build AS publish
RUN dotnet publish "UnitTestApp/UnitTestApp.csproj" -c Release -o /app
RUN dotnet publish "UnitTestApp.Tests/UnitTestApp.Tests.csproj" -c Release -o /app

# Запуск тестов
RUN dotnet test "UnitTestApp.Tests/UnitTestApp.Tests.csproj"

FROM publish AS final
WORKDIR /app COPY --from=publish /app .
ENTRYPOINT ["dotnet", "UnitTestApp.dll"]
```

- **FROM** — задаёт базовый (родительский) образ. Мы использовали официальный образ .NET Core SDK[1].
- **ORKDIR** — задаёт рабочую директорию для следующей инструкции.
- **EXPOSE** — указывает на необходимость открыть порт.
- **COPY** — копирует в контейнер файлы и папки.
- **RUN** — выполняет команду и создаёт слой образа. Используется для установки в контейнер пакетов.
- **ENTRYPOINT** — предоставляет команду с аргументами для вызова во время выполнения контейнера. Аргументы не переопределяются.

Если в проекте всего одно приложение, то на данном этапе можно остановиться, и начать сборку с последующим развертыванием собранного контейнера на сервере.

Находясь в корне проекта, командой:
$ docker build –file UnitTestApp.Dockerfile.
мы можем запустить сборку контейнера, а затем развернуть его на сервере.
Но мы будем считать, что в будущем проект будет пополнятся различными микросервисами. И в ручную запускать сборку каждого Dockerfile не удобно, на помощь приходит **docker-compose.**

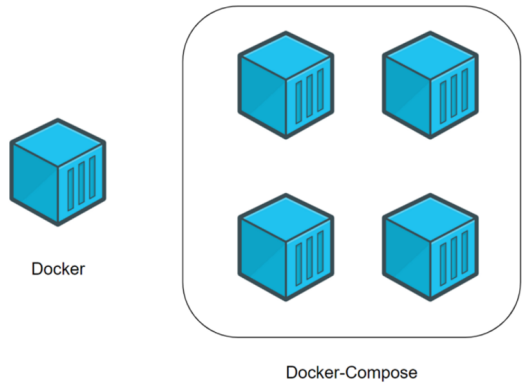

Рис. 10 docker-compose

Docker Compose существует поверх докера и просто позволяет управлять контейнерами на основе единого конфигурационного файла, в котором описаны контейнеры, их параметры и их взаимосвязи (например контейнер A имеет право соединяться с портом 5432 контейнера B)

Давайте создадим в корне проекта файл *docker-compose.yml* и наполним его следующим содержимым:

```
# Файл docker-compose должен начинаться с тега версии.
# Мы используем "3" так как это - самая свежая версия
#на момент написания этого кода.
version: '3.4'

# Следует учитывать, что docker-composes работает с сервисами.
# 1 сервис = 1 контейнер.
# Сервисом может быть клиент, сервер, сервер баз данных...
# Раздел, в котором будут описаны сервисы, начинается с 'services'.
services:

 # Здесь мы, для именования соответствующего сервиса, используем
 # ключевое слово 'unittestapp'.
 unittestapp:
    # Здесь мы указываем какое название будет иметь собранный
```

```
#Docker образ
# переменная TAG определена в файле .env в той же
# директории что и данный файл
image: "unittestapp:\${TAG}"
# Ключевое слово "build" позволяет задать
# путь к файлу Dockerfile, который нужно использовать для
# создания образа,
# который позволит запустить сервис.
build:
    # Здесь './' соответствует пути к директории сервиса
    # "UnitTestsApp"
    context: ./
    # Наименование Dockerfile расположенного по пути
    # указанном в "context"
    dockerfile: UnitTestApp.Dockerfile
```

Достаточно трудноуловимой ошибкой при сборке контейнеров, является верхний регистр в docker-compose файле, в частности, присвоение имени контейнеру после сборки (поле image)

Содержимое файла .env:

TAG = staging

Теперь для запуска сборки образов контейнеров, описанных в файле *docker-compose.yml* выполним следующую команду:

docker-compose build

Команда *docker-compose build* по умолчанию ищет в текущей директории файл с названием *docker – compose.yml* и производит сборку контейнеров. В случае, если файл Docker Compose назван по другому, следует его указать, явно, следующей командой:

$ docker-compose –file NameDockerComposeFile build,

где **NameDockerComposeFile** путь к вашему Docker Compose файлу.

Во время сборки будет выводиться информация о каждом собранном сервисе:

```
Step 13/17 : RUN dotnet test "UnitTestApp.Tests/UnitTestApp.Tests.csproj"
 ---> Running in 8732a1322b67
Test run for /src/UnitTestApp.Tests/bin/Debug/netcoreapp3.1/UnitTestApp.Tests.dl
l(.NETCoreApp,Version=v3.1)
Microsoft (R) Test Execution Command Line Tool Version 16.5.0
Copyright (c) Microsoft Corporation.  All rights reserved.

Starting test execution, please wait...

A total of 1 test files matched the specified pattern.

Test Run Successful.
Total tests: 20
     Passed: 20
 Total time: 3.1984 Seconds
Removing intermediate container 8732a1322b67
 ---> f61cb2399a87

Step 14/17 : FROM mcr.microsoft.com/dotnet/core/aspnet:3.1 AS runtime
 ---> c819eb4381e7
Step 15/17 : WORKDIR /app
 ---> Using cache
 ---> 84ab495d9590
Step 16/17 : COPY --from=publish /app .
 ---> 0f2dc67e1193
Step 17/17 : ENTRYPOINT ["dotnet", "UnitTestApp.dll"]
 ---> Running in 442ea8005041
Removing intermediate container 442ea8005041
 ---> a59a22a67c31

Successfully built a59a22a67c31
Successfully tagged unittestapp:latest
```

Рис. 11 Вывод информации о каждом собранном сервисе

3.4 Загрузка собранного контейнера в репозиторий Docker контейнеров

Список локальных собранных сервисов можно узнать командой:
$ docker image ls

```
$ docker image ls
REPOSITORY                                  TAG                 IMAGE ID            CREATED             SIZE
unittestapp                                 latest              a59a22a67c31        About an hour ago   252MB
bi                                          latest              a3cb6ecf1738        8 days ago          1.21GB
newunittestsapp_bi                          latest              a50dbe2eb694        8 days ago          1.21GB
newunittestsapp_web                         latest              b1080b3ac2b7        10 days ago         1.21GB
pyramidadapter                              latest              76ca34852d18        11 days ago         1.75GB
```

Рис. 12 Список локальных собранных сервисов

Для хранения своих собранных образов необходимо их где то хранить, на помощь приходят репозитории Docker.

Например:**DockerHub**.

Но в данном курсе мы не будем его использовать, т.к. мы не хотим делиться своими разработками с миром, а на момент написание данной методички, у него не имеется бесплатных приватных репозиториев.

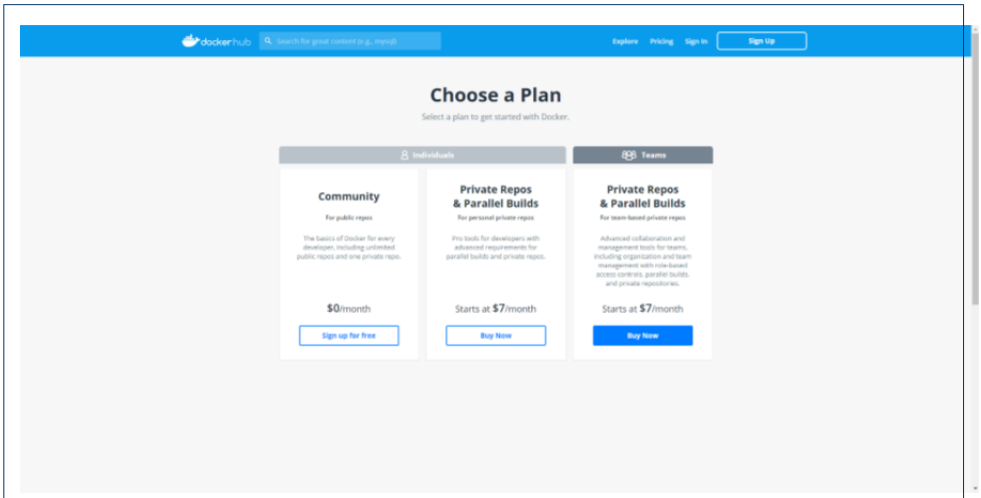

Рис. 13 CDockerHub

Мы будем использовать сервис реестр контейнеров **TreeScale**, но у него есть небольшие ограничения:

1 Не более 500 pull в месяц
2 Объем реестра не должен превышать 50 ГБ
3 Не более 50 сборок образов в месяц

После регистрации на TreeScale[10], необходимо авторизоваться в реестре контейнеров пр помощи Docker, как написано в официально приложении Б, командой:

$ docker login repo.treescale.com

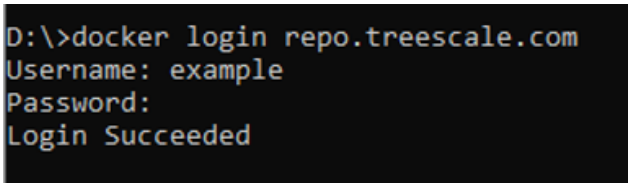

Рис. 14 Команда

Затем, для загрузки контейнеров в репозиторий TreeScale необходимо что бы собранные контейнеры имели префикс *repo.treescale.com/[username]/*, а полное наименование собранного контейнера выглядело так:
repo.treescale.com/[username]/[image name]:[tag name]

Изменим файл docker-compose следующим образом:

```
version: '3.4'

services:
    unittestapp:
        # Добавили префикс "repo.treescale.com/[username]/"
        image: "repo.treescale.com/Example/:\${TAG}"
```

```
    build:
        context: ./
        dockerfile: UnitTestApp.Dockerfile
```

После успешной сборки контейнера:

```
Step 14/17 : FROM mcr.microsoft.com/dotnet/core/aspnet:3.1 AS runtime
 ---> c819eb4381e7
Step 15/17 : WORKDIR /app
 ---> Using cache
 ---> 84ab495d9590
Step 16/17 : COPY --from=publish /app .
 ---> Using cache
 ---> 0f2dc67e1193
Step 17/17 : ENTRYPOINT ["dotnet", "UnitTestApp.dll"]
 ---> Using cache
 ---> a59a22a67c31

Successfully built a59a22a67c31
Successfully tagged repo.treescale.com/example/unittestapp:latest
```

Рис. 15 Успешная сборка контейнеров

необходимо создать на сервисе TreeScale новый реестр.
Переходим на страницу [10] с нашими реестрами и нажимаем **ADD NEW REGISTRY**

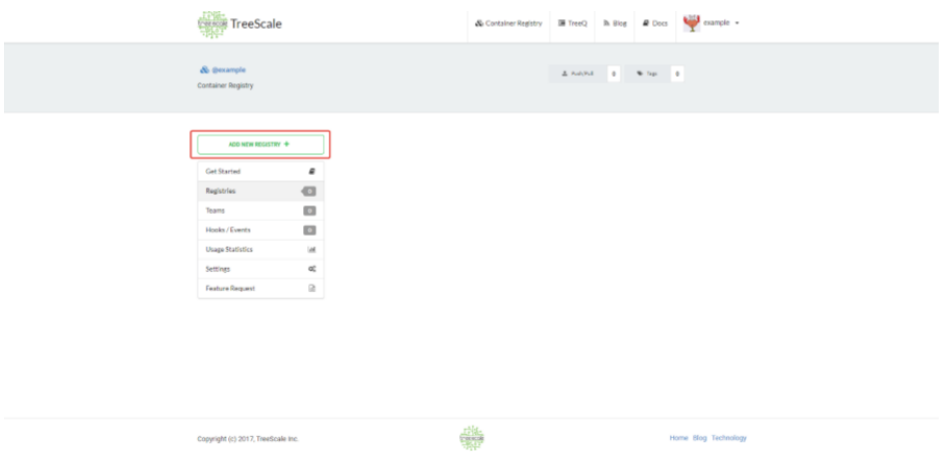

Рис. 16 Страница с реестрами

В поле **REGISTRY NAME** необходимо указать значение **[image name]** указанное в полном названии собранного контейнера *repo.treescale.com/[username]/[image name]:[tag name]*:

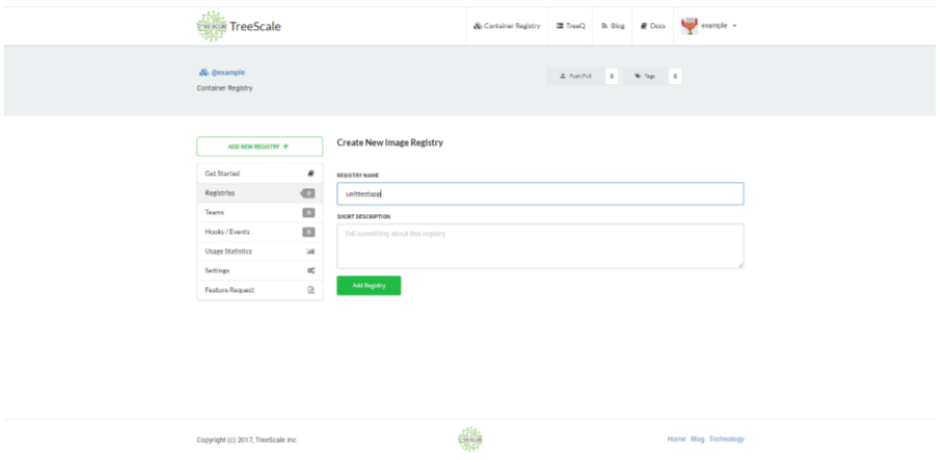

Рис. 17 Ввод значения

Командой *$ docker push ImageName* загружаем собранный образ в реестр контейнеров:

```
$ docker push repo.treescale.com/example/unittestapp:latest
The push refers to repository [repo.treescale.com/example/unittestapp]
94ee9a33c577: Preparing
1e7025a28e9e: Preparing
4de7fcec79e4: Preparing
b4f12c28bcb9: Preparing
4be6dadfbd3f: Preparing
b50dd2025901: Preparing
c3a984abe8a8: Preparing
b50dd2025901: Waiting
c3a984abe8a8: Waiting
1e7025a28e9e: Pushed
4be6dadfbd3f: Pushed
4de7fcec79e4: Pushed
94ee9a33c577: Pushed
b4f12c28bcb9: Pushed
b50dd2025901: Pushed
c3a984abe8a8: Pushed
latest: digest: sha256:9cec9c2cc528a5315ca85de245627d07e979ef0b10c2fc760528b24371a11d94 size: 1794
```

Рис. 18 Командная строка

Мы успешно загрузили собранный образ в реестр контейнеров !

3.5 Контрольные вопросы

Список вопросов для отчета:

1. Что такое Docker и зачем он нужен?
2. Что хранит в себе Dockerfile
3. Кратко опишите процесс создания Docker образа и его загрузку в реестр контейнеров.
4. Почему было исключено использование DockerHub?
5. Кратко опишите процесс создания Docker образа и его загрузку в реестр контейнеров.

4 Лабораторная работа №4

Наименование: Развертывание проекта на кластере

4.1 Цели и задачи

Целью работы является получение навыков работы с кластером и практическое применение навыков на примере разрабатываемого приложения.

Целью работы является получение навыков работы с кластером и практическое применение навыков на примере разрабатываемого приложения.

1 Целью работы является получение навыков работы с кластером и практическое применение навыков на примере разрабатываемого приложения.
2 Изучение и применение механизмов развертывания ИТ-систем:
   2.1 Развертывание системы с помощью Ansible;
   2.2 Развертывание системы с помощью TFS Piplines.

4.2 Критерии приемки работ

1 Знание основных команд работы с кластером;
2 Успешное развертывание системы с помощью Ansible;
3 Успешное развертывание системы с помощью TFS Piplines.

4.3 Работа с кластером (основные команды)

4.3.1 Развертывание системы с помощью Ansible

Инструкция по развертыванию проекта из Git репозитория отображена в приложении B.

4.3.2 Unit-тесты при развертывании

Для этого необходимо в Dockerfile написать инструкцию для запуска Unit-тестов при сборке.

Для C# проекта:

```
...
RUN dotnet publish "UnitTestApp.Tests/UnitTestApp.Tests.csproj"
-c Release -o /app


# Запуск тестов
RUN dotnet test "UnitTestApp.Tests/UnitTestApp.Tests.csproj"


FROM base AS final
...
```

Для Python (Django) проекта:

```
...
 EXPOSE 8000
```

```
# Запуск тестов
RUN python manage.py test


CMD [ "python", "manage.py", "runserver", "0.0.0.0:8000" ]
...
```

4.4 Ручной запуск Unit-тестов в Docker контейнере

4.4.1 Проект на C#

Для запуска Unit-тестов в ручную, нужно:

1. Подключиться к запущенному Docker контейнеру: *$ docker exec -it NAME_UP_CONTAINER bash,*
   где NAME_UP_CONTAINER - название запущенного контейнера (можно узнать командой $ docker ps).

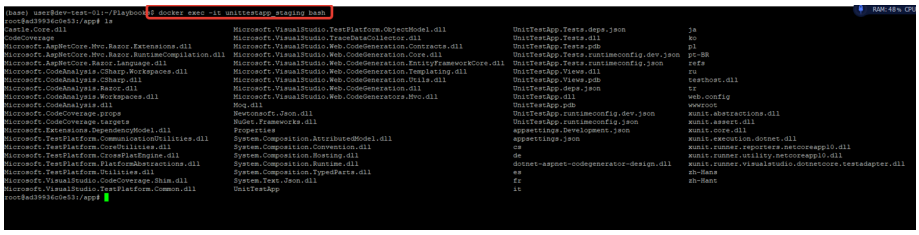

Рис. 19 Командная строка

2. Запустить Unit-тесты, указав dll файл проекта содержащего Unit-тесты *dotnet test NAME_PROJECT.Tests.dll*, где NAME_PROJECT - название проекта
   Вывод успешного прохождения всех тестов:

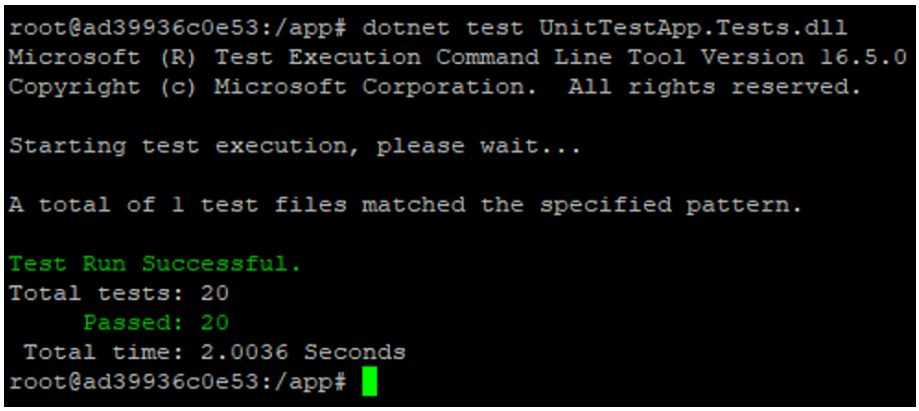

Рис. 20 Вывод успешного прохождения тестов

4.4.2 Проект на Python (Django)

Для запуска Unit-тестов в ручную, нужно:

1 Подключиться к запущенному Docker контейнеру: *$ docker exec -it NAME_UP_CONTAINER bash,* NAME_UP_CONTAINER - название запущенного контейнера (можно узнать командой $ docker ps)

```
(base) user@dev-test-01:~/Playbooks$ docker exec -it bi_staging bash
root@675580ef229c:/# ls
BI  Data.csv  Dockerfile  Stemy_BI  bin  boot  db.sqlite3  dev  docker-biserver-ce  docker-compose.yml  et
root@675580ef229c:/#
```

Рис. 21 Командная строка

2 **Запустить Unit-тесты**

Запускаем файл *manage.py* и пердаем в него единственный аргумент test *$ python manage.py test* Вывод успешного прохождения всех тестов:

```
user@dev-test-01: ~/Playbooks
(base) user@dev-test-01:~/Playbooks$ docker exec -it bi_staging bash
root@675580ef229c:/# ls
BI  Data.csv  Dockerfile  Stemy_BI  bin  boot  db.sqlite3  dev  docker-biserver-ce  doc
root@675580ef229c:/# python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
../usr/local/lib/python3.6/site-packages/django/db/models/fields/__init__.py:1423: Runt
  RuntimeWarning)
...../usr/local/lib/python3.6/site-packages/django/db/models/fields/__init__.py:1423
  RuntimeWarning)
./usr/local/lib/python3.6/site-packages/django/db/models/fields/__init__.py:1423: Runti
  RuntimeWarning)
./usr/local/lib/python3.6/site-packages/django/db/models/fields/__init__.py:1423: Runti
  RuntimeWarning)
.
----------------------------------------------------------------------
Ran 13 tests in 4.619s

OK
Destroying test database for alias 'default'...
root@675580ef229c:/#
```

Рис. 22 Вывод успешного прохождения тестов

4.5 Развертывание системы с помощью TFS Piplines

4.6 Контрольные вопросы

Список вопросов для отчета:

1 Приведите пример команд для работы с кластером.
2 Что такое Playbooks и зачем они нужны?
3 Опишите процесс работы с Ansible.
4 Опишите механизмы развертывания ИТ-систем.
5 Перечислите основные команды для работы с кластером.

Список литературы

1. ASP.NET [Электронный ресурс]. – [2020]. – Режим доступа : https://dotnet.microsoft.com/apps/aspnet
2. Python [Электронный ресурс]. – [2020]. – Режим доступа : https://www.python.org/doc/ https://dotnet.microsoft.com/apps/aspnet
3. Documentation [Электронный ресурс]. – [2020]. – Режим доступа : https://docs.djangoproject.com/en/3.0/
4. TDDx2, BDD, DDD, FDD, MDD и PDD, или все, что вы хотите узнать о Driven Development [Электронный ресурс]. – [2020]. – Режим доступа : https://habr.com/ru/post/459620/
5. Тестирование. Фундаментальная теория [Электронный ресурс]. – [2020]. – Режим доступа : https://habr.com/ru/post/279535/
6. Osherove R. The Art of Unit Testing. – Manning Publications Co: Shelter Island, 2014.
7. GitHub [Электронный ресурс]. – [2020]. – Режим доступа: https://github.com/LaboratoryBilab/UnitTests_Asp.Net-Core
8. GitHub [Электронный ресурс]. – [2020]. – Режим доступа : https://github.com/LaboratoryBilab/UnitTestsDjango/tree/master/Stemy_BI
9. GitHub [Электронный ресурс]. – [2020]. – Режим доступа: https://github.com/LaboratoryBilab/UnitTestsDjango
10. РД 50-34.698-90 Методические указания. Информационная технология. Комплекс стандартов и руководящих документов на автоматизированные системы. Автоматизированные системы. Требования к содержанию документов [Электронный ресурс]. – [2020]. – Режим доступа : http://docs.cntd.ru/document/1200006978
11. TreeScale [Электронный ресурс]. – [2020]. – Режим доступа : https://treescale.com/

Приложение А

Образец протокола

Тестирование и оценка качества систем Лабораторная работа №1 Выбор предметной области	ФИО студента	Иванов Иван Иванович
	Группа	ИВТ-463
	Преподаватель	Аль-Гунаид Мохаммед Амин
	Дата	
	Оценка	

Цели работы

Познакомиться с сервером баз данных MS SQL Server 2008, изучить основные типы данных, используемых в MS SQL. Получить практические навыки создания баз данных с помощью утилиты SQL Server Management Studio. Получить практические навыки создания многостраничных веб-приложений с использованием фреймворка ASP.NETMVC и работы с БД с помощью Entity Framework.

Постановка задачи

Необходимо в среде SQL Server Management Studio реализовать базу данных «Аэропорт» и веб-интерфейс с использованием технологии ASP.NET для работы с таблицами данных.

Ход работы

Здесь следует привести:

1. Архитектуру системы.
2. Функциональную структуру системы.
3. Скриншоты веб-страниц.
4. Код C#-скриптов, реализующих базовый набор операций с таблицами данных.

Выводы

В результате лабораторной работы получены практические навыки:

1) создания баз данных с помощью утилиты SQL Server Management Studio;

2) создания многостраничных веб-приложений с использованием фреймворка ASP.NET MVC и работы с БД с помощью Entity Framework.

Также, в среде SQL Server Management Studio, реализована база данных «Аэропорт» и разработан с использованием фреймворка ASP.NET веб-интерфейс, реализующий базовый набор операций с таблицами данных.

Приложение Б

Требования и рекомендации по оформлению кода

Требования

Структура проекта, именование файлов

- Не следует размещать в одном файле больше одного класса (за исключением, возможно, вложенных типов и типов делегатов);
- В отсутствии противопоказаний имя файла должно совпадать с именем определенного в нем класса;
- Структура директорий должна быть максимально приближена к структуре пространств имен.

Pascal casing

Описываются имена:

- всех определений типов, в том числе пользовательских классов, перечислений, событий, делегатов и структур;
- значения перечислений;
- публичных полей;
- констант;
- интерфейсов;
- методов;
- пространств имен (namespace);
- свойств.

```
namespace SampleNamespace
{
  enum SampleEnum
  {
    FirstValue,
    SecondValue
  }

  struct SampleStruct
  {
    public int FirstField;
    public int SecondField;
  }

  interface ISampleInterface
  {
    void SampleMethod();
  }
```

```
  public class SampleClass: SampleInterface
  {
    const int SampleConstValue = 0xffffff;

    readonly int SampleReadonlyField;

    public int SampleProperty
    {
      get;
      set;
    }

    public int SamplePublicField;

    SampleClass()
    {
      SampleReadonlyField = 1;
    }

    delegate void SampleDelegate();
    event SampleDelegate SampleEvent;

    public void SampleMethod()
    {
    }
  }
}
```

Camel casing

Описываются имена:

- локальных переменных;
- аргументов методов;
- защищенных (protected) полей.

```
protected int sampleProtectedField;

int SampleMethod(int sampleArgument)
{
  int sampleLocalVariable;
}
```

Использование var

Переменные с одновременным присваиванием в теле методов должны быть объявлены как var.

```
var errorMessage = new StringBuilder();
```

Если тип из правой части назначения не является очевидным, не рекомендуется использовать var.

Суффиксы и префиксы

Применяются следующие суффиксы и префиксы:

- имена пользовательских классов исключений всегда заканчиваются суффиксом “Exception”;
- имена интерфейсов всегда начинаются с префикса «I»;
- имена пользовательских атрибутов всегда заканчиваются суффиксом «Attribute»;
- имена делегатов обработчиков событий всегда оканчиваются суффиксом EventHandler, имена классов-наследников от EventArgs всегда заканчиваются суффиксом EventArgs.

```
public class SampleException: System.Exception
{
  public SampleException()
  {
  }
}

interface ISample
{
  void SampleMethod();
}

[System.AttributeUsage(System.AttributeTargets.All, Inherited = false, AllowMultiple =
true)]
sealed class SampleAttribute: System.Attribute
{
  public SampleAttribute()
  {
  }
}
public delegate void AnswerCreatedEventHandler(object sender, AnswerCreatedEventArgs e);
public class AnswerCreatedEventArgs: EventArgs
{
   public int CreatedId;
   public int ParentId;
   public string CreatorName;
}
```

Приложение В

Программа и методика испытаний информационной системы

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«ВОЛГОГРАДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(ВолгГТУ)

им. В.И.Ленина пр-кт, 28, г. Волгоград, 400005, Телефон 23-00-76, факс 23-41-21

e-mail: rector@vstu.ru

http://www.vstu.ru

ОГРН 1023403440818

ОКПО 02068060

ИНН/КПП 3444049170/344401001

Версия N

«Тема»

Научный руководитель (Преподаватель)

к.т.н., Аль-Гунаид М.А. _____

Ответственный исполнитель (Студент)

Группа

ФИО _____

Волгоград 202*

Аннотация

Настоящая Программа и методика испытаний информационной системы …….

Программа и методика испытаний разработана в соответствии с требованиями ГОСТ 34.603-92, РД 50-34.698-90.

Содержание

1 Объект испытаний 4
1.1 Наименование системы 4
1.2 Область применения системы 4
1.3 Условное обозначение системы 4
2 Цель испытаний 4
3 Общие положения 4
3.1 Перечень руководящих документов, на основании которых проводятся испытания 4
3.2 Место и продолжительность испытаний 5
3.3 Организации, участвующие в испытаниях 5
3.4 Перечень предъявляемых на испытания документов 5
4 Объем испытаний 5
4.1 Перечень этапов испытаний и проверок в соответствии с положениями технического проекта (ТП) и требованиями технического задания (ТЗ) 5
4.2 Испытания информационной базы данных XXXXX 6
4.3 Испытания модуля сбора и обработки данных 6
5 Методика проведения испытаний 7
6 Требования по испытаниям программных средств 8
7 Перечень работ, проводимых после завершения испытаний 8
8 Условия и порядок проведения испытаний 8
9 Материально-техническое обеспечение испытаний 9
10 Метрологическое обеспечение испытаний 10
11 Отчётность 10

1 Объект испытаний

1.1 Наименование системы

Наименование информационной системы – XXXXXX

1.2 Область применения системы

ИС предназначена для инструментальной поддержки функционирования XXXXXXXX

1.3 Условное обозначение системы

Условное обозначение (краткое наименование) информационной системы: XXXXX.

2 Цель испытаний

Целью проводимых по настоящей программе и методике испытаний XXXX является определение функциональной работоспособности системы на этапе проведения испытаний.

Программа испытаний должна удостоверить работоспособность XXXX в соответствии с функциональным предназначением.

3 Общие положения

3.1 Перечень руководящих документов, на основании которых проводятся испытания

Приемочные испытания XXXX проводятся на основании следующих документов:

- XXXXX (Лабораторная работа № 1);
- Настоящая Программа и методика испытаний;

3.2 Место и продолжительность испытаний

Место проведения испытаний – площадка Заказчика

Продолжительность испытаний устанавливается Приказом Заказчика о составе приемочной комиссии и проведении приемочных испытаний.

3.3 Организации, участвующие в испытаниях

В приемочных испытаниях участвуют представители следующих организаций:

- XXXXXXXX

- Федеральное государственное бюджетное образовательное учреждение высшего образования «Волгоградский государственный технический университет» (ФГБОУ ВО «ВолгГТУ»).

Конкретный перечень лиц, ответственных за проведение испытаний системы, определяется Заказчиком.

3.4 Перечень предъявляемых на испытания документов

Для проведения испытаний Исполнителем предъявляются следующие документы:

- Техническое задание на создание XXXX;

4 Объем испытаний

4.1 Перечень этапов испытаний и проверок в соответствии с положениями технического проекта (ТП) и требованиями технического задания (ТЗ)

В процессе проведения приемочных испытаний должны быть протестированы следующие модули XXXXX:

4.1.1 Информационная база данных XXXX;

4.1.2 Модуль сбора и обработки данных;

Все подсистемы испытываются одновременно на корректность взаимодействия подсистем, влияние подсистем друг на друга, т.е. испытания проводятся комплексно.

Приемочные испытания включают проверку:

- полноты и качества реализации функций, указанных в ТЗ;
- выполнения каждого требования, относящегося к интерфейсу Системы;
- работы пользователей в диалоговом режиме;
- полноты действий, доступных пользователю, и их достаточность для функционирования Системы;
- реакции системы на ошибки пользователя.

4.2 Испытания информационной базы данных XXXXX

Испытания информационной базы данных XXXX направлены на проверку наличия доступа пользователей к базе данных внутри локальной сети.

Требования к информационной базе данных XXXX и оборудования приведены в Техническом задании.

4.3 Испытания модуля сбора и обработки данных

Испытания модуля сбора и обработки данных направлены на проверку работоспособности следующих функций:

- добавление, просмотр, редактирование и удаление данных;
- просмотр и редактирование данных, необходимых для выполнения алгоритмов.

5 Методика проведения испытаний

Таблица 1. Методика проведения испытаний

№	Действие	Результат
1.	**Сценарий «Работа с данными» (пункт 4.1.1, 4.1.2)**	
1.1	Войти в систему как пользователь с правами на создание данных.	Новое экземпляр данных создан в ИБД
1.2	Войти в систему как пользователь с правами на редактирование данных.	Внесенные изменения сохранены.
1.5	Войти в систему как пользователь с правами на удаление данных.	Внесенные изменения сохранены.
2	**Сценарий «Обработка данных в ИБД» (пункт 4.1.2)**	
2.1	Выполнить расчет по имеющимся данным в ИБД.	Расчет успешно выполнен и сохранен в ИБД.
2.2	Выгрузка данных по расчетам.	Данные успешно выгружены.
2.3	Удалить один из имеющихся расчетов.	Выбранный расчет успешно удален из ИБД.
3	**Сценарий «Работа с пользователями» (пункт 4.1.2)**	

3.1	Войти в систему как пользователь с правами на управление пользователями, добавить нового пользователя с необходимым набором прав.	Новый пользователь успешно добавлен в систему.
3.2	Редактировать данные одного из существующих пользователей.	Изменения указанного пользователя вступили в силу.
3.3	Удалить пользователя.	Пользователь удален из списка.

6 Требования по испытаниям программных средств

Испытания программных средств XXX проводятся в процессе функционального тестирования Информационной Системы и её нагрузочного тестирования (п. 4.2).

Других требований по испытаниям программных средств XXXXXX не предъявляется.

7 Перечень работ, проводимых после завершения испытаний

По результатам испытаний делается заключение о соответствии XXXXXX требованиям ТЗ на Систему и возможности оформления акта сдачи XXXXXX в опытную эксплуатацию. При этом производится (при необходимости) доработка программных средств и документации.

8 Условия и порядок проведения испытаний

Испытания XXXXXX должны проводиться на целевом оборудовании Заказчика. Оборудование должно быть предоставлено в той конфигурации,

которая запланирована для начального развертывания системы, и указана в Техническом задании.

Во время испытаний проводится полное функциональное тестирование, согласно требованиям, указанным в Техническом задании.

При проведении приемочных испытаний доступ к функционалу Системы предоставляется ограниченному кругу пользователей.

В ходе проведения опытной эксплуатации для каждого зарегистрированного пользователя Системы администратор определяет разделы Системы, к которым данный пользователь получит доступ для проведения полнофункционального тестирования. В момент авторизации осуществляется проверка роли и полномочий пользователя, в зависимости от которых пользователю предоставляется доступ к определённым разделам Системы, а также определяется набор функционала, отвечающий задачам данного пользователя.

Данные пользователи работают с Системой, выполняя свои служебные обязанности, то есть размещают, редактируют, публикуют и удаляют содержание, подвергая тем самым XXXXXX полнофункциональному тестированию в течение установленного срока.

9 Материально-техническое обеспечение испытаний

Приемочные испытания проводятся на программно-аппаратном комплексе Заказчика в следующей минимальной конфигурации:

Серверная площадка:

- Операционная система – Windows 7 и выше, Windows Server 2012 и выше;
- Сетевое подключение – подключение по протоколу TCP;
- MS SQL Server Express 2014, или MS SQL Server 2014:

Рабочее место:

- Операционная система – Windows 7 и выше;

- Сетевое подключение – подключение по протоколу TCP;

10 Метрологическое обеспечение испытаний

Программа испытаний не требует использования специализированного измерительного оборудования.

11 Отчётность

Результаты испытаний XXXXXX, предусмотренные настоящей программой, фиксируются в протоколах, содержащих следующие разделы:

- Назначение испытаний и номер раздела требований ТЗ на XXXXXX, по которому проводят испытание;

- Состав технических и программных средств, используемых при испытаниях;

- Указание методик, в соответствии с которыми проводились испытания, обработка и оценка результатов;

- Условия проведения испытаний и характеристики исходных данных;

- Средства хранения и условия доступа к тестирующей программе;

- Обобщенные результаты испытаний;

- Выводы о результатах испытаний и соответствии созданной Системы определённому разделу требований ТЗ на XXXXXX.